

Trabajo de Laboratorio

Semáforo Inteligente

Sistemas de Tiempo Real - Ingeniería en Informática
Universidad Nacional de San Luis

2026

1. Instrucciones generales

- Completar este trabajo en grupos de 3 a 4 personas.
- Entregar un informe en PDF antes de la fecha indicada por la cátedra que incluya:
 - el programa NuSMV para cada una de versiones solicitadas,
 - las propiedades,
 - las capturas de pantalla de los resultados, y
 - las respuestas a las preguntas.

El informe debe evidenciar que los alumnos lograron:

- La identificación de requisitos funcionales y temporales.
- El modelado del sistema como una máquina de estados finitos con temporizadores.
- Especificación de propiedades en lógica temporal CTL y LTL.
- Verificación automática y análisis de contraejemplos.

2. Semáforo inteligente

Un semáforo inteligente debe responder a la demanda de vehículos y peatones, sin dejar de respetar restricciones temporales estrictas. En una intersección de calles con eje Norte-Sur (NS) y Este-Oeste (EO), cada acceso cuenta con un semáforo de tres colores (verde, amarillo, rojo), un semáforo peatonal de dos colores (naranja y blanco) y un sensor de presencia de vehículos que indica si hay al menos un vehículo esperando. Las aceras además cuentan con dos botones peatonales.

Se sugiere usar las variables de estado que muestra el código debajo.

```
1  MODULE main
2  VAR
3  sem_ns  : {verde, amarillo, rojo};
4  sem_eo  : {verde, amarillo, rojo};
5  timer_ns : 0..TM;
6  timer_eo : 0..TM;
```

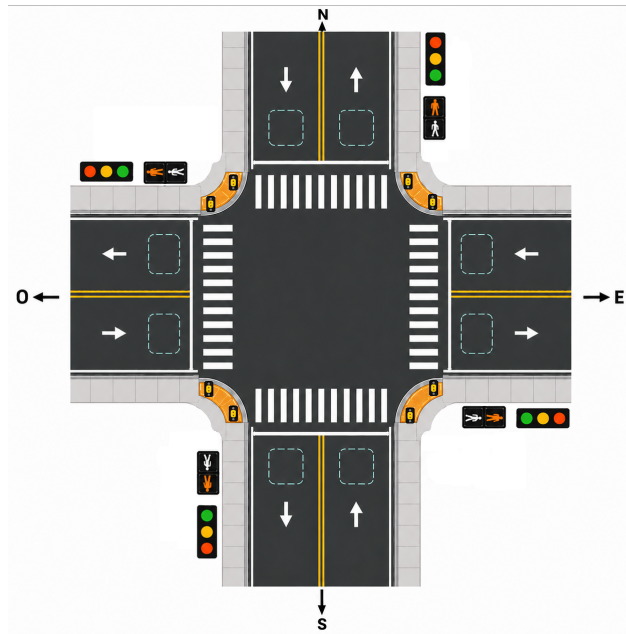


Figura 1: Semáforo inteligente

```

7  demanda_ns : boolean;
8  demanda_eo : boolean;

```

Donde TM es una constante que define el tiempo máximo del temporizador (por ej. 60 ticks).

El comportamiento del sistema debe respetar que:

- φ_1 : Los semáforos de direcciones cruzadas no pueden estar en verde simultáneamente.
- φ_2 : El amarillo debe durar exactamente T_{amarillo} y no debe saltarse.
- φ_3 : Si hay vehículos esperando en una dirección y el semáforo está rojo, eventualmente se pondrá verde.
- φ_4 : El tiempo de verde no debe exceder un máximo T_{max} (ej. 30 segundos).
- φ_5 : Siempre es posible que algún semáforo esté en verde.

3. Actividades

1. **Modelo base cíclico fijo.** Implementar en NuSMV un modelo donde los semáforos cambian de fase únicamente basados en temporizadores (sin considerar demanda). Elegir valores adecuados para $T_{\text{verde}} = 10$ ticks, $T_{\text{amarillo}} = 3$ ticks. Verificar si las propiedades se cumplen.
2. **Demanda y extensión inteligente.** Modificar el modelo para que el semáforo se mantenga en verde si hay demanda en su mismo sentido, hasta un máximo de $T_{\text{max}} = 15$ ticks. Si no hay demanda en el sentido actual pero sí en el otro, el cambio de fase ocurre inmediatamente (respetando el amarillo). Verificar las propiedades nuevamente y si alguna falla, explicar por qué.

3. **Espera del vehículo.** Modificar el modelo para poder verificar que el tiempo máximo de espera para un vehículo (desde que llega hasta que obtiene verde) no supere $L = 20$ ticks. Especificar esta propiedad en CTL y verificarla.
4. **Botón peatonal.** Incorporar al modelos los botones peatonales. Si el botón peatonal es presionado, lo cual puede ocurrir en cualquier momento, el semáforo debe dar verde peatonal (simulado como una fase especial donde ambos semáforos están en rojo) dentro de los siguientes 10 ticks.
5. **Contraejemplos.** Modificar el modelo para introducir errores deliberados que violen cada una de las propiedades establecidas. Analizar el contraejemplo generado por NuSMV.
6. **Escalabilidad y rendimiento** Para el modelo final (con demanda y botón peatonal), medir el tiempo de verificación y el número de estados alcanzables usando el comando `print_reachable_states` en NuSMV. Comparar con el modelo cíclico fijo. Discutir la complejidad del espacio de estados.
7. **Preguntas conceptuales (responder en el informe)**
 - (a) ¿Por qué es importante verificar propiedades de seguridad en sistemas de tiempo real? Dé un ejemplo concreto diferente al de los semáforos.
 - (b) Explique la diferencia entre CTL y LTL.
 - (c) ¿Cómo modelaría un fallo en el sensor de demanda (ej. sensor siempre activo)? ¿Qué propiedad se violaría?